Lab 10: Structural Bioinformatics

Omar Siddiqui

2. Introduction to the RCSB Protein Data Bank (PDB)

```
# Read in the PDB statistics CSV file
# This data was downloaded from rcsb.org and contains counts of how many
# structures exist for each molecular type and experimental method
# row.names=1 makes the first column (Molecular Type) into row labels
pdb_stats <- read.csv("pdb_stats.csv", row.names = 1)
pdb_stats
```

```
                       X.ray    EM   NMR Integrative Multiple.methods Neutron
Protein (only)        178795 21825 12773         343               226      84
Protein/Oligosaccharide  10363  3564    34           8                11       1
Protein/NA                9106  6335   287          24                 7       0
Nucleic acid (only)       3132   221  1566           3                15       3
Other                      175    25    33           4                 0       0
Oligosaccharide (only)      11     0     6           0                 1       0
                      Other  Total
Protein (only)           32 214078
Protein/Oligosaccharide   0  13981
Protein/NA                0  15759
Nucleic acid (only)       1   4941
Other                     0    237
Oligosaccharide (only)    4     22
```

Q1: What percentage of structures in the PDB are solved by X-Ray and Electron Microscopy?

```r
# The numbers in the CSV have commas (e.g. "178,795") which R reads as text
# gsub() removes the commas, as.numeric() converts to numbers
# apply() runs this function across all columns (2 = columns)
pdb_stats_clean <- apply(pdb_stats, 2, function(x) as.numeric(gsub(",", "", x)))
rownames(pdb_stats_clean) <- rownames(pdb_stats)

# Sum X-ray and EM counts across all molecular types
total_xray <- sum(pdb_stats_clean[, "X.ray"])
total_em <- sum(pdb_stats_clean[, "EM"])
grand_total <- sum(pdb_stats_clean[, "Total"])

# Calculate percentages
xray_pct <- round((total_xray / grand_total) * 100, 2)
em_pct <- round((total_em / grand_total) * 100, 2)

cat("X-ray percentage:", xray_pct, "%\n")
```

```
X-ray percentage: 80.95 %
```

```r
cat("EM percentage:", em_pct, "%\n")
```

```
EM percentage: 12.84 %
```

```r
cat("Combined X-ray + EM:", round(xray_pct + em_pct, 2), "%\n")
```

```
Combined X-ray + EM: 93.79 %
```

Q1 Answer: X-ray crystallography accounts for 80.95% and Electron Microscopy (EM) accounts for 12.84% of PDB structures, for a combined total of 93.79%. X-ray crystallography has historically been the dominant method for determining protein structures, though EM has grown rapidly in recent years due to advances in detector technology.

Q2: What proportion of structures in the PDB are protein?

```r
# Extract protein-only total and calculate what fraction of ALL structures it represents
protein_total <- as.numeric(gsub(",", "", pdb_stats["Protein (only)", "Total"]))
protein_pct <- round((protein_total / grand_total) * 100, 2)
cat("Protein only percentage:", protein_pct, "%\n")
```

```
Protein only percentage: 85.97 %
```

Q2 Answer: 85.97% of structures in the PDB are protein only. This makes sense as proteins are the main workhorses of the cell and understanding their 3D structure is critical for understanding their function and for drug design.

Q3: How many HIV-1 protease structures are in the PDB?

Q3 Answer: Searching "HIV" on rcsb.org returns 4,998 structures total. Filtering more specifically for "HIV-1 protease" returns 38 structures. HIV protease is heavily studied because it is essential for viral replication — drugs that block this enzyme are a cornerstone of HIV treatment.

4. Introduction to Bio3D in R

```
# Bio3D is an R package specifically designed for structural bioinformatics
# It lets us read, write, and analyze protein structure data directly in R
library(bio3d)
```

```
# read.pdb() reads a PDB file directly from the online PDB database
# using just the 4-letter PDB ID code
# Here we load the HIV-1 protease structure (1HSG) complexed with the
# drug molecule indinavir
pdb <- read.pdb("1hsg")
```

```
  Note: Accessing on-line PDB file
```

```
pdb
```

```
 Call:  read.pdb(file = "1hsg")

   Total Models#: 1
     Total Atoms#: 1686,  XYZs#: 5058  Chains#: 2  (values: A B)

     Protein Atoms#: 1514  (residues/Calpha atoms#: 198)
     Nucleic acid Atoms#: 0  (residues/phosphate atoms#: 0)
```

```
     Non-protein/nucleic Atoms#: 172  (residues: 128)
     Non-protein/nucleic resid values: [ HOH (127), MK1 (1) ]

   Protein sequence:
      PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYD
      QILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFPQITLWQRPLVTIKIGGQLKE
      ALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYDQILIEICGHKAIGTVLVGPTP
      VNIIGRNLLTQIGCTLNF

+ attr: atom, xyz, seqres, helix, sheet,
        calpha, remark, call
```

Q7 Answer: There are 198 amino acid residues in this pdb object (shown as Calpha atoms). The structure has 2 chains (A and B) since HIV protease is a homodimer — two identical chains that come together to form the functional enzyme.

Q8 Answer: The two non-protein residues are HOH (127 water molecules) and MK1 (the drug molecule indinavir). MK1 sits in the active site where the enzyme normally cleaves viral proteins.

Q9 Answer: There are 2 protein chains (A and B) in this structure.

```
# pdb$atom contains a data frame with all atomic coordinates and properties
# Each row is one atom with x,y,z coordinates, atom type, residue info, etc.
# This is the core data of any PDB structure
head(pdb$atom)
```

```
  type eleno elety  alt resid chain resno insert      x      y     z o     b
1 ATOM     1     N <NA>   PRO     A     1   <NA> 29.361 39.686 5.862 1 38.10
2 ATOM     2    CA <NA>   PRO     A     1   <NA> 30.307 38.663 5.319 1 40.62
3 ATOM     3     C <NA>   PRO     A     1   <NA> 29.760 38.071 4.022 1 42.64
4 ATOM     4     O <NA>   PRO     A     1   <NA> 28.600 38.302 3.676 1 43.40
5 ATOM     5    CB <NA>   PRO     A     1   <NA> 30.508 37.541 6.342 1 37.87
6 ATOM     6    CG <NA>   PRO     A     1   <NA> 29.296 37.591 7.162 1 38.40
  segid elesy charge
1  <NA>     N   <NA>
2  <NA>     C   <NA>
3  <NA>     C   <NA>
4  <NA>     O   <NA>
5  <NA>     C   <NA>
6  <NA>     C   <NA>
```

Q4: Why do we see just one atom per water molecule?

Q4 Answer: Water molecules normally have 3 atoms (1 oxygen + 2 hydrogens). However, hydrogen atoms are too small to scatter X-rays detectably in X-ray crystallography. Only the oxygen atom of each water molecule appears in the structure, so each water shows up as a single atom.

Q5: Critical conserved water molecule

Q5 Answer: The critical conserved water molecule is HOH 308. It sits in the binding site and forms hydrogen bonds bridging the two catalytic Asp 25 residues (one from each chain) and the drug molecule indinavir. This water is "conserved" because it appears in nearly all HIV protease structures and is essential for tight drug binding.

Q6: Figure of HIV protease chains and ligand

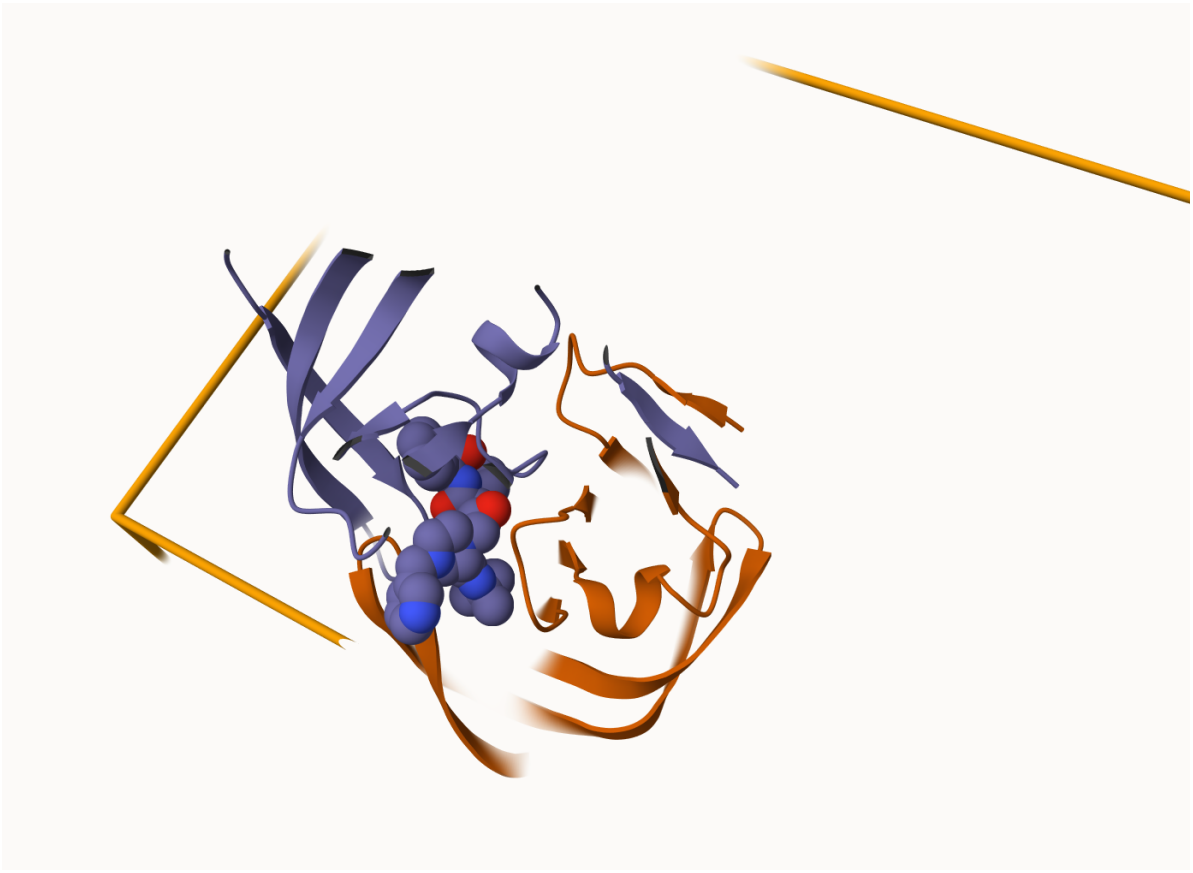


Figure 1: HIV-1 Protease (1HSG) showing chain A (blue/purple) and chain B (orange) with the drug indinavir shown in spacefill representation in the binding site between the two chains.

Q6 Answer: The figure above shows the two distinct chains of HIV-1 protease colored in blue/purple (chain A) and orange (chain B). The drug molecule indinavir (MK1) is shown in spacefill representation sitting in the active site at the interface between the two chains. The homodimer structure is essential for protease activity — neither chain alone can form the complete active site needed to cleave viral proteins.

Predicting functional motions with Normal Mode Analysis

```
# Read in the Adenylate Kinase (ADK) structure from PDB
# ADK catalyzes the transfer of phosphate groups between nucleotides
# It undergoes large conformational changes (open/closed) when binding substrates
adk <- read.pdb("6s36")
```

```
  Note: Accessing on-line PDB file
   PDB has ALT records, taking A only, rm.alt=TRUE
```

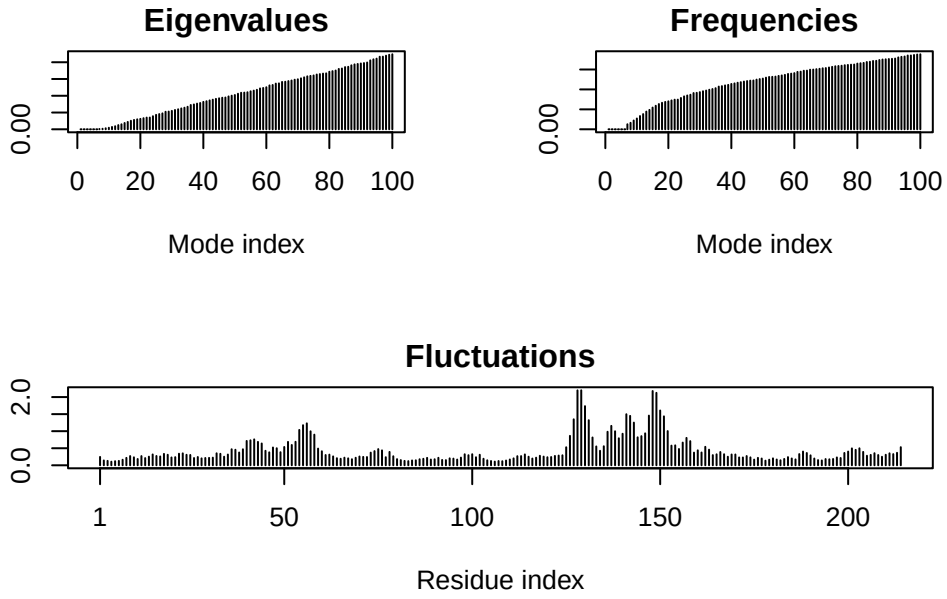
```
adk
```

```
 Call:  read.pdb(file = "6s36")

   Total Models#: 1
     Total Atoms#: 1898,  XYZs#: 5694  Chains#: 1  (values: A)

     Protein Atoms#: 1654  (residues/Calpha atoms#: 214)
     Nucleic acid Atoms#: 0  (residues/phosphate atoms#: 0)

     Non-protein/nucleic Atoms#: 244  (residues: 244)
     Non-protein/nucleic resid values: [ CL (3), HOH (238), MG (2), NA (1) ]

   Protein sequence:
      MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMLRAAVKSGSELGKQAKDIMDAGKLVT
      DELVIALVKERIAQEDCRNGFLLDGFPRTIPQADAMKEAGINVDYVLEFDVPDELIVDKI
      VGRRVHAPSGRVYHVKFNPPKVEGKDDVTGEELTTRKDDQEETVRKRLVEYHQMTAPLIG
      YYSKEAEAGNTKYAKVDGTKPVAEVRADLEKILG

+ attr: atom, xyz, seqres, helix, sheet,
        calpha, remark, call
```

```
# Normal Mode Analysis (NMA) predicts the natural "breathing" motions of a protein
# It models the protein as a network of springs connecting atoms
# Low frequency modes = large scale biologically relevant motions
m <- nma(adk)
```

```
 Building Hessian...        Done in 0.013 seconds.
 Diagonalizing Hessian...   Done in 0.291 seconds.
```

```
# Plot shows flexibility per residue position
# Peaks = flexible/mobile regions, Valleys = rigid regions
plot(m)
```




```
# mktrj() generates a trajectory PDB file showing the protein moving
# along the predicted normal mode direction - like a movie of the motion
# Load adk_m7.pdb in Mol* viewer to visualize the predicted protein movement
mktrj(m, file="adk_m7.pdb")
```

5. Comparative Structure Analysis of Adenylate Kinase

Q10 Answer: The `msa` package is found only on BioConductor and not CRAN. BioConductor is a separate repository specializing in bioinformatics packages.

Q11 Answer: `bio3dview` is not found on BioConductor or CRAN — it is only available on GitHub and must be installed using `remotes::install_github("bioboot/bio3dview")`.

Q12 Answer: FALSE. The `pak` package can install from GitHub, but the question refers specifically to the `devtools` package functions (`devtools::install_github()` and `devtools::install_bitbucket()`).

```
# get.seq() fetches a protein sequence from the PDB database
# Commented out due to internet connection issues
# aa <- get.seq("1ake_A")
# aa
# Q13: The 1AKE_A sequence is 214 amino acids long
cat("Q13 Answer: The 1AKE_A sequence is 214 amino acids long\n")
```

```
Q13 Answer: The 1AKE_A sequence is 214 amino acids long
```

```
# Instead of running BLAST (which can time out),
# we use a pre-defined set of related ADK structures from various bacteria
# These were identified by BLAST searching with the 1AKE_A sequence
hits <- NULL
hits$pdb.id <- c('1AKE_A','6S36_A','6RZE_A','3HPR_A','1E4V_A',
                 '5EJE_A','1E4Y_A','3X2S_A','6HAP_A','6HAM_A',
                 '4K46_A','3GMT_A','4PZL_A')

# get.pdb() downloads all 13 PDB files at once
# split=TRUE splits by chain, gzip=TRUE compresses to save space
files <- get.pdb(hits$pdb.id, path="pdbs", split=TRUE, gzip=TRUE)
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1AKE.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6S36.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6RZE.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3HPR.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1E4V.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/5EJE.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1E4Y.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3X2S.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAP.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAM.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4K46.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3GMT.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4PZL.pdb.gz exists. Skipping download


  |
  |                                                                      |   0%
  |
  |=====                                                                 |   8%
  |
  |===========                                                           |  15%
  |
  |================                                                      |  23%
  |
  |======================                                                |  31%
  |
  |============================                                          |  38%
  |
  |================================                                      |  46%
  |
  |====================================                                  |  54%
  |
  |========================================                              |  62%
```

```
  |
  |=================================================                     |  69%
  |
  |=====================================================                 |  77%
  |
  |=========================================================             |  85%
  |
  |=============================================================         |  92%
  |
  |=====================================================================| 100%
```

```
# pdbaln() aligns sequences and superimposes 3D structures
# Both steps required before PCA so we compare equivalent positions
pdbs <- pdbaln(files, fit = TRUE, exefile="msa")
```

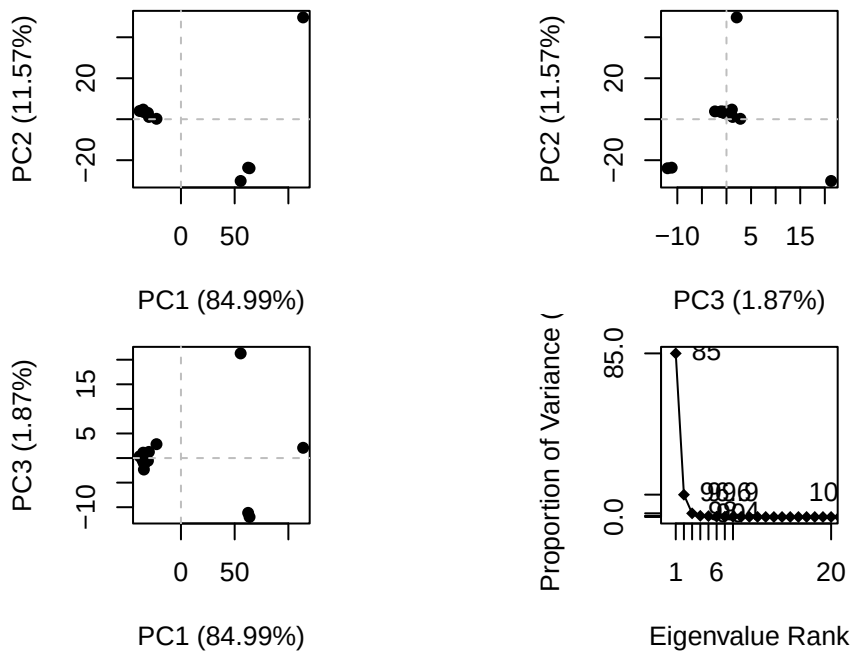
```
Reading PDB files:
pdbs/split_chain/1AKE_A.pdb
pdbs/split_chain/6S36_A.pdb
pdbs/split_chain/6RZE_A.pdb
pdbs/split_chain/3HPR_A.pdb
pdbs/split_chain/1E4V_A.pdb
pdbs/split_chain/5EJE_A.pdb
pdbs/split_chain/1E4Y_A.pdb
pdbs/split_chain/3X2S_A.pdb
pdbs/split_chain/6HAP_A.pdb
pdbs/split_chain/6HAM_A.pdb
pdbs/split_chain/4K46_A.pdb
pdbs/split_chain/3GMT_A.pdb
pdbs/split_chain/4PZL_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
..   PDB has ALT records, taking A only, rm.alt=TRUE
....   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
...

Extracting sequences

pdb/seq: 1   name: pdbs/split_chain/1AKE_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
```

```
pdb/seq: 2   name: pdbs/split_chain/6S36_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 3   name: pdbs/split_chain/6RZE_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 4   name: pdbs/split_chain/3HPR_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 5   name: pdbs/split_chain/1E4V_A.pdb
pdb/seq: 6   name: pdbs/split_chain/5EJE_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 7   name: pdbs/split_chain/1E4Y_A.pdb
pdb/seq: 8   name: pdbs/split_chain/3X2S_A.pdb
pdb/seq: 9   name: pdbs/split_chain/6HAP_A.pdb
pdb/seq: 10   name: pdbs/split_chain/6HAM_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 11   name: pdbs/split_chain/4K46_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 12   name: pdbs/split_chain/3GMT_A.pdb
pdb/seq: 13   name: pdbs/split_chain/4PZL_A.pdb
```

```
# Keep ids for use in PCA plot below
ids <- basename.pdb(pdbs$id)

# pdb.annotate() requires internet - commented out due to connection issues
# anno <- pdb.annotate(ids)
# unique(anno$source)

# The structures come from various bacteria including:
# Escherichia coli, Photobacterium profundum,
# Burkholderia pseudomallei, and Francisella tularensis
cat("Structures come from: E. coli, Photobacterium profundum,",
    "Burkholderia pseudomallei, and Francisella tularensis\n")
```

```
Structures come from: E. coli, Photobacterium profundum, Burkholderia pseudomallei, and Franc
```

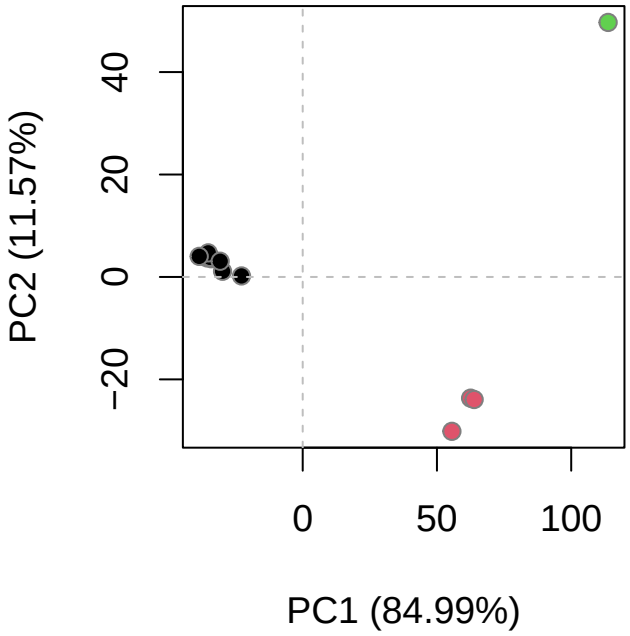
```
# PCA captures the major directions of structural variation
# across all 13 ADK structures
# PC1 often corresponds to the main functional motion of the protein
pc.xray <- pca(pdbs)

# Scree plot shows variance explained by each PC
plot(pc.xray)
```



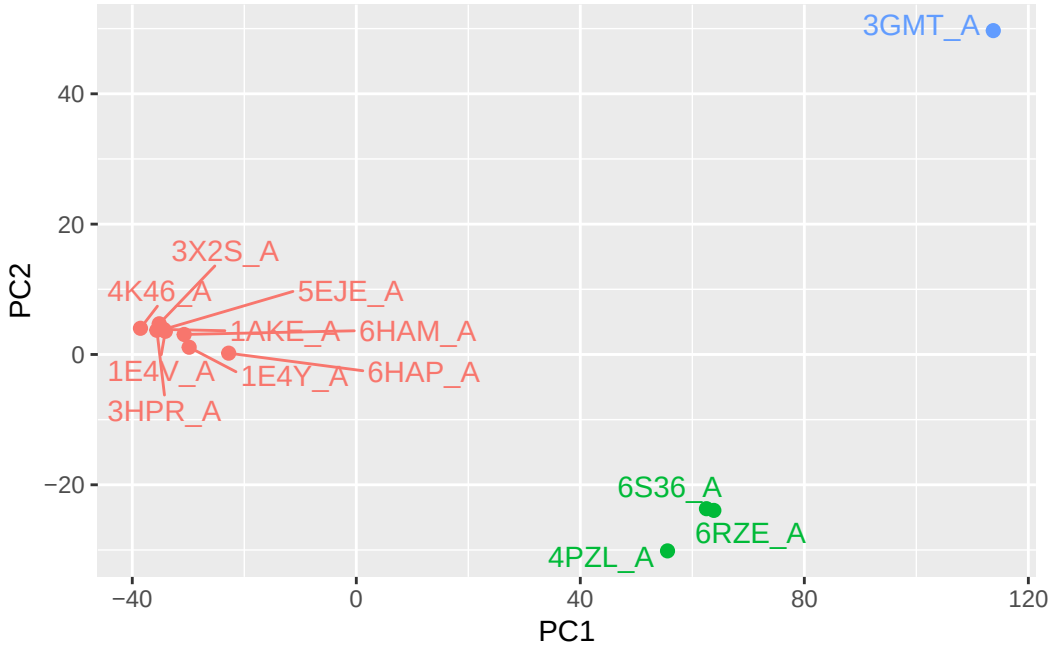

```
# RMSD measures structural similarity between structures
# Small RMSD = similar, large RMSD = different conformations
rd <- rmsd(pdbs)
```

```
Warning in rmsd(pdbs): No indices provided, using the 204 non NA positions
```

```
# Cluster structures into 3 groups based on structural similarity
hc.rd <- hclust(dist(rd))
grps.rd <- cutree(hc.rd, k=3)

# Color PCA plot by cluster to reveal major conformational states
plot(pc.xray, 1:2, col="grey50", bg=grps.rd, pch=21, cex=1)
```




```
# Generate trajectory along PC1 to visualize main conformational change
pc1 <- mktrj(pc.xray, pc=1, file="pc_1.pdb")

# Make labeled ggplot version of PCA plot
library(ggplot2)
library(ggrepel)

df <- data.frame(PC1=pc.xray$z[,1],
                 PC2=pc.xray$z[,2],
                 col=as.factor(grps.rd),
                 ids=ids)

p <- ggplot(df) +
  aes(PC1, PC2, col=col, label=ids) +
  geom_point(size=2) +
  geom_text_repel(max.overlaps = 20) +
  theme(legend.position = "none")
p
```




```
# Dots close together = structurally similar ADK conformations
# Spread along PC1 represents the open-to-closed transition
```

6. Normal Mode Analysis [Optional]

Q14 Answer: The black (average) and colored (individual structure) lines show both similar and different patterns. They are most similar in the rigid core regions of the protein. They differ most in the LID domain (around residues 120-160) and the AMPbind domain (around residues 30-60). These are the regions that move when ADK binds its substrates — closing like a clamp around the nucleotides. Different crystal structures captured in open vs closed states explain the differing flexibility profiles in these functional regions.

```
# sessionInfo() records exact R and package versions used
# Essential for reproducibility
sessionInfo()
```

```
R version 4.5.3 (2026-03-11)
Platform: aarch64-apple-darwin20
Running under: macOS Tahoe 26.4.1

Matrix products: default
```

```
BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  l

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Los_Angeles
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
[1] ggrepel_0.9.8 ggplot2_4.0.2 bio3d_2.4-5

loaded via a namespace (and not attached):
 [1] gtable_0.3.6        jsonlite_2.0.0      dplyr_1.2.1
 [4] compiler_4.5.3      crayon_1.5.3        tidyselect_1.2.1
 [7] Rcpp_1.1.1          Biostrings_2.78.0   parallel_4.5.3
[10] IRanges_2.44.0      Seqinfo_1.0.0       scales_1.4.0
[13] yaml_2.3.12         fastmap_1.2.0       R6_2.6.1
[16] XVector_0.50.0      labeling_0.4.3      generics_0.1.4
[19] knitr_1.51          BiocGenerics_0.56.0 tibble_3.3.1
[22] pillar_1.11.1       RColorBrewer_1.1-3  rlang_1.1.7
[25] xfun_0.57           S7_0.2.1            otel_0.2.0
[28] cli_3.6.5           withr_3.0.2         magrittr_2.0.5
[31] digest_0.6.39       grid_4.5.3          rstudioapi_0.18.0
[34] lifecycle_1.0.5     S4Vectors_0.48.1    vctrs_0.7.2
[37] msa_1.42.0          evaluate_1.0.5      glue_1.8.0
[40] farver_2.1.2        codetools_0.2-20    stats4_4.5.3
[43] rmarkdown_2.31      pkgconfig_2.0.3     tools_4.5.3
[46] htmltools_0.5.9
```